APOLLO: SGD-like Memory, AdamW-level Performance

Ying Jin Felipe Vicentin

1 Introduction

Nowadays, AdamW is still the default optimizer for Transformer training because its adaptive first and second moment statistics stabilize optimization and outperform purely first order methods such as SGD [1, 2]. Ignoring weight decay, AdamW can be expressed as follows:

$$M_t = \beta_1 M_{t-1} + (1-\beta_1)G_t, \quad V_t = \beta_2 V_{t-1} + (1-\beta_2)G_t^2$$
$$W_{t+1} = W_t - \eta \cdot \frac{M_t}{\sqrt{V_t} + \varepsilon} \tag{1}$$

where M_t, V_t are first/second moments, G_t the gradient, W_t weights, $\beta_{1,2}$ decay rates, η learning rate, and ε a stability constant. However, this stability comes at the cost of a memory overhead. In large-scale training, the optimizer states alone consume twice as much memory as the model parameters. To address this memory-intensive nature of second-order moments, APOLLO integrates two core concepts:

- **Coarse grained adaptivity**: the adaptive scale can be shared across a channel or a whole tensor, and
- **LowRank Updates**: Statistics can be maintained in a low rank subspace via random projection.

Firstly, one can see that AdamW's update rule is fundamentally element-wise

$$W_{t+1} = W_t - \eta \cdot \widetilde{G}_t = W_t - \eta \cdot S_t \odot G_t, \quad S_t = \widetilde{G}_t \oslash G_t \tag{2}$$

where $\tilde{G}_t = \frac{M_t}{\sqrt{V_t+\epsilon}}$, S_t the adaptive scale, and $\odot, \oslash$ denote element-wise division.
Recent work on optimizer redundancy, especially Adam-mini, argues that block or group wise second moment statistics can be sufficient for Transformer training [3]. In other words, Transformers do not always require a fully independent adaptive scale for every parameter [2]. The core hypothesis is that this scaling rule is often more fine-grained than necessary. Instead of assigning one scale per parameter, one may share a scale across an entire channel(row or column) or even an entire tensor:

$$\widetilde{G}_t^{\text{channel}} = G_t \cdot \text{diag}(s_t), \; s_t[j] = \frac{\|\widetilde{G}_t[:,j]\|_2}{\|G_t[:,j]\|_2}; \quad \widetilde{G}_t^{\text{tensor}} = s_t^{\text{tensor}} G_t, \; s_t^{\text{tensor}} = \frac{\|\widetilde{G}_t\|_2}{\|G_t\|_2} \tag{3}$$

Said differently, the structured update keeps the raw gradient direction but replaces the element-wise adaptive scale with a coarser channel-level or tensor-level factor.
That said, structured scaling by itself does not solve the memory problem. To compute s_t exactly, we still need access to the full AdamW moments. Low-rank gradient optimization was already explored by methods such as GaLore and Fira [6], which reduce optimizer-state memory by updating projected gradients [4]. The main drawback is that these methods commonly rely on singular value decomposition (SVD) to build the subspace which are costly. Here comes the second idea: randomized projections. Specifically, we project G_t to rank $r \ll m$ and estimate s_j in that space:

$$R_t = P_t G_t \in \mathbb{R}^{r \times n}, \qquad s_j^R = \frac{\|\tilde{R}_t[:,j]\|_2}{\|R_t[:,j]\|_2} \approx s_j \cdot \sqrt{\frac{r}{n}}$$

Then, we have the following expressions for the randomly projected first and second moments:

$$M_t^R = \beta_1 M_{t-1}^R + (1-\beta_1)R_t, \qquad V_t^R = \beta_2 V_{t-1}^R + (1-\beta_2)R_t^2$$

The resulting channel-wise scale is then estimated from the low-rank statistics instead of the full-rank ones. In the extreme tensor-wise rank-1 case of APOLLO-Mini, the memory cost becomes comparable to SGD-level state storage, because then we have one auxiliary variable per tensor.
The Apollo's theoretical support lies in a sequence of approximation guarantees showing that random projection preserves the key norms needed to estimate the structured scaling factors using Johnson-Lindenstrauss (JL) Lemma [7] [8]. If the projected gradients preserve the channel-wise momentum norms sufficiently well, then the resulting low-rank scaling factors remain close to those of the full-rank structured optimizer [5]. This is the key justification for replacing SVD

Optimizer	best_eval_perplexity ↓	Memory
AdamW	3.61	0.4327 GB
APOLLO rank-1/4	4.13	0.1082 GB
APOLLO rank-1/8	4.13	0.0541 GB
apollomini	53.66	0.0009 GB

Table 1: Perplexities come from the TinyStories LLaMA-60M runs with optimizers : AdamW, Apollo rank 1/4, Apollo rank 1/8, apollomini.

with random projection: APOLLO does not try to reconstruct the full gradient exactly, but only the coarse adaptive scaling needed to modulate the gradient update.
We want to prove whether an APOLLO-like optimizer can reduce optimizer-state memory while keeping training quality reasonably close to AdamW.
This project aligns with the core challenges of large-scale model training, specifically addressing the critical hardware-software bottlenecks inherent in distributed systems. In modern multi-GPU environments, training efficiency is increasingly memory-bound rather than compute-bound. APOLLO offers a decisive system-level advantage by slashing the VRAM footprint of optimizer states, thereby creating significant headroom for larger global batch sizes and increased throughput. By reducing static memory overhead, APOLLO better amortizes the communication and synchronization costs across distributed ranks, minimizing the need for costly memory offloading or re-computation. Ultimately, this work explores a fundamental shift in optimization: moving from a singular focus on algorithmic convergence to holistic system efficiency, enabling the training of larger architectures within fixed hardware constraints.

2 Model Setup

We focus on pre-training studies instead of fine-tuning as pre-training from scratch can better reveal the stability of optimizers. In this sense, we conduct three experiments:

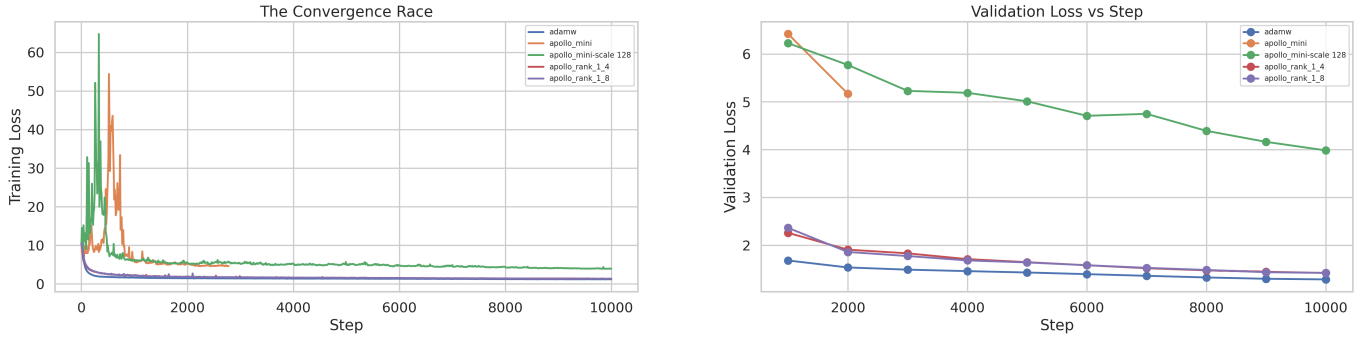
1. LLaMA-60M on TinyStories; optimizers: AdamW, APOLLO rank-1/4, APOLLO rank-1/8, APOLLO-Mini.
2. NanoGPT-40M on TinyStories; optimizers: AdamW, APOLLO rank-1/4, naive channel-wise, and group-wise variants with $K = 2, 4$.
3. NanoGPT-130M on WikiText; optimizers: AdamW and channel-wise structured scaling.

We use validation perplexity as the main metric for evaluting pretraining performance of the model with optimizers.

3 Results

3.1 Partial success:LLaMA-60M + TinyStories

Fig. 1 and Tab. 1 show the results are close enough to the paper's core claim to count as a meaningful validation in a reduced setting.
Both APOLLO rank-1/4 and rank-1/8 demonstrate stable convergence, tracking the AdamW baseline closely while reducing optimizer-state memory linearly with their respective ranks. The 10k-step duration provides a sufficient horizon to confirm this stability.
Conversely, APOLLO-Mini (rank-1) exhibits significant instability, characterized by severe early loss spikes and a markedly inferior final loss. Even after tuning the scaling hyperparameters (128 and 16), the rank-1 variant consistently suffers from volatile initial training and persistent underperformance.
Fig. 2(c) illustrates how the APOLLO series reduces optimizer-state memory (shown in red), with APOLLO-Mini achieving a memory footprint at SGD level. Our Out-of-Memory (OOM) stress tests further reveal that AdamW's maximum runnable batch size is marginally lower than those of the APOLLO variants. Regarding computational efficiency, AdamW maintains a slight advantage in both batch throughput and per-step latency, which aligns with its slightly superior pre-training performance. Indeed, in small-scale settings, the system-level advantages of APOLLO—specifically its ability to bypass memory-bound constraints—are not yet fully realized.



(a) Training Loss

(b) Validation Loss

Figure 1: LLaMA-60M + TinyStories with AdamW, APOLLO rank-1/4 and rank-1/8, APOLLO-mini

Optimizer	best_eval_perplexity
AdamW(element-wise)	5.23
grouped_rowwise_k2	11.59
grouped_rowwise_k4	11.22
Channel-wise (no random projection)	14.10

Table 2: Perplexities from the TinyStories nanogpt-40M after 8k steps with element-wise, grouped rowwise_k2, grouped_rowwised_k4, channel-wise without random projection

3.2 Sanity check on the coarsility of gradients on NanoGPT Architecture

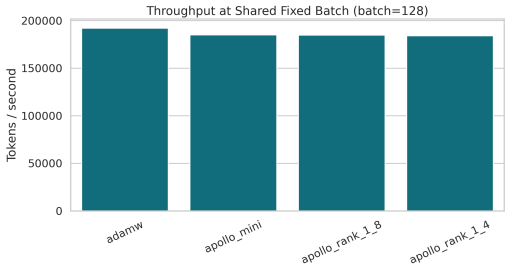
APOLLO exploits parameter redundancy by hypothesizing that adaptive scales can be shared across channels or tensors with minimal performance loss. This coarse-grained scaling is a prerequisite; without its validation, employing random projections to emulate AdamW's adaptivity lacks justification.

While the original study empirically validated these claims on LLaMA-130M, our experiments on NanoGPT-40M reveal a performance divergence. As shown in Fig. 3b and Tab. 2, deterministic channel-wise scaling (without random projection) fails to approach AdamW's baseline, with a perplexity of 14.10 compared to 5.23. To investigate if the granularity of scaling was the primary bottleneck, we tested group-wise sharing with $K = 2$ and $K = 4$. While these variants showed marginal improvements over the channel-wise approach, they remained substantially inferior to the element-wise update. We extended our evaluation to NanoGPT-130M on the WikiText dataset. As illustrated in Figure 4, while the model exhibits rapid overfitting, the persistent performance gap between channel-wise and element-wise scaling reconfirms the failure of our mechanistic sanity check. This suggests that the insufficiency of structured adaptivity is not a localized issue but a systemic characteristic of the NanoGPT architecture.

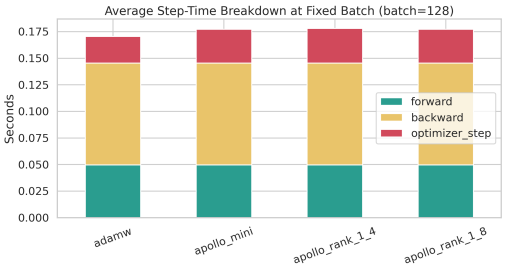
These suggest that for this NanoGPT architecture trained on Tinystories, fine-grained, element-wise adaptivity is indispensable for stable and efficient convergence.

3.3 Sanity Check on APOLLO's Correction factor

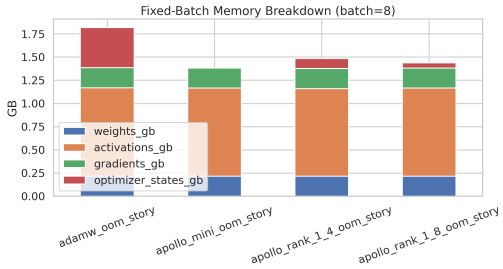
The theoretical framework of APOLLO can be interpreted as recovering full-rank adaptive scales from a low-rank subspace using a $\sqrt{n/r}$ correction factor, under the premise that random projections preserve the essential norm of the gradient. However, our empirical analysis reveals that for APOLLO-$n/4$ in the Llama experiment, the observed ratio, $\frac{\|\mathbf{g}_{low}\|}{\|\mathbf{g}_{full}\|}$ falls consistently below the theoretical lower bound of 1/2 (derived from $\sqrt{r/n}$). This discrepancy suggests that the low-rank projection fails to capture sufficient spectral energy of the gradient, leading to substantial information loss during the reconstruction process. This lack of gradient fidelity provides a mechanistic explanation for the marginal performance gap observed between APOLLO and AdamW. Furthermore, we observe a progressive decay of the reconstruction ratio in deeper layers, with the empirical value $\frac{\|\mathbf{g}_{low}\|}{\|\mathbf{g}_{full}\|}$ asymptotically approaching zero. This phenomenon might suggest a gradient rank collapse in the later stages of the architecture. As the gradient energy concentrates into an increasingly narrow intrinsic subspace, the data-agnostic nature of random projections leads to a 'misalignment' between the projection matrix and the principal gradient directions. Consequently, the fixed scaling factor $\sqrt{n/r}$ fails to compensate for the massive spectral energy loss, exacerbating the optimization lag in deeper layers.



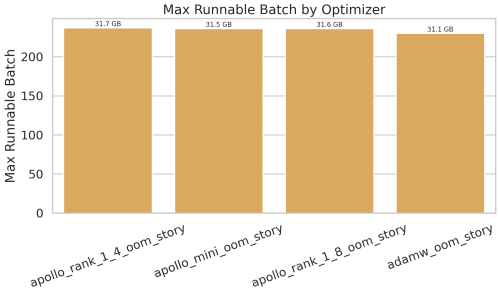
(a) Batch Throughput



(b) Step Time Breakdown



(c) Memory Breakdown



(d) Max Runnable Batch

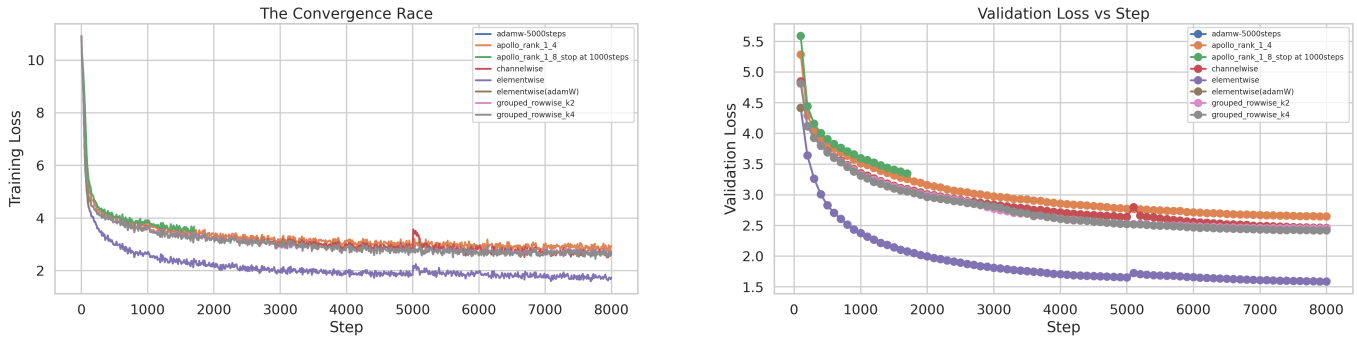
Figure 2: Memory analysis for LLaMA-60M: (a, b) display the throughput and per-step time consumption; (c, d) illustrate the memory footprint and the maximum achievable batch size under OOM stress.

4 Discussion

The rank ablation in Experiment 1 demonstrates that rank-1/8 yields nearly identical perplexity to rank-1/4 on the LLaMA-60M architecture. This suggests that structured scaling estimates are remarkably robust to extreme compression, aligning with the theoretical claim that the $\sqrt{n/r}$ correction factor is effectively absorbed into the learning rate, keeping the approximation error bounded even at ultra-low ranks. For practitioners, this is a significant finding: optimizer-state memory can be halved again with negligible performance degradation.APOLLO-Mini (rank-1), however, presents a distinct challenge. By employing tensor-wise scaling—where a single scalar modulates all weight updates within a tensor—it introduces an extreme coarsening relative to channel-wise methods. In our reduced setup, rank-1 appears insufficiently expressive to estimate reliable adaptive scales during the volatile early stages of training, resulting in severe loss spikes. While the original paper achieves stability by fine-tuning hyperparameters (notably the scale factor α and learning rate), our attempts to stabilize training with scaling factors of 128 and 16 were unsuccessful. This suggests that for this specific configuration, the norm-growth limiter alone is inadequate to compensate for the information loss inherent in rank-1 approximations.

On the memory side, APOLLO delivers on its core promise: optimizer-state memory scales proportionally to rank, as expected from the $2nr$ state cost. However, in the small setting experiment, the memory analysis tell a honest aligned with the result. We expect Apollo's advancatge truns into memory reducing, thus leading to larger batch size, higher throughout, allowing longer context length, the story is not compelling on the small setting where on single-GPU runs with small models, the workload is often compute-bound rather than memory-bound, so reducing optimizer states does not directly translate into higher tokens per second. A fair throughput comparison would require running at the same memory budget, scaling batch size up to exploit APOLLO's freed capacity, a regime closer to the multi-GPU, large-batch setting where the original paper reports its strongest gains.

The most salient finding of this project is the stark architectural divergence in response to structured scaling: while LLaMA-60M demonstrates robust compatibility, the NanoGPT series (40M and 130M) exhibits consistent performance degradation across both TinyStories and WikiText. Our group-wise ablation studies ($K = 2$ and $K = 4$) also reveals even marginal coarsening of the scaling factor across just two parameters impairs model quality. This suggests that the assumption of scale-compressibility, which underpins the APOLLO framework, is invalid for the standard NanoGPT architecture. Unlike LLaMA, NanoGPT appears to necessitate high-fidelity, element-wise adaptive statistics, likely due to a higher degree of gradient anisotropy that coarse-grained updates fail to capture. Our hypothesis is that APOLLO's efficacy depends on how compressible the adaptive scaling rule is for a given architecture. LLaMA uses RMSNorm and RoPE, which may regularize activation and gradient statistics in ways that make channel-wise scaling more coherent



(a) Training Loss

(b) Validation Loss

Figure 3: nanogpt-40M + TinyStories with element-wise, channel-wise, group-wise with $k = 2, 4$

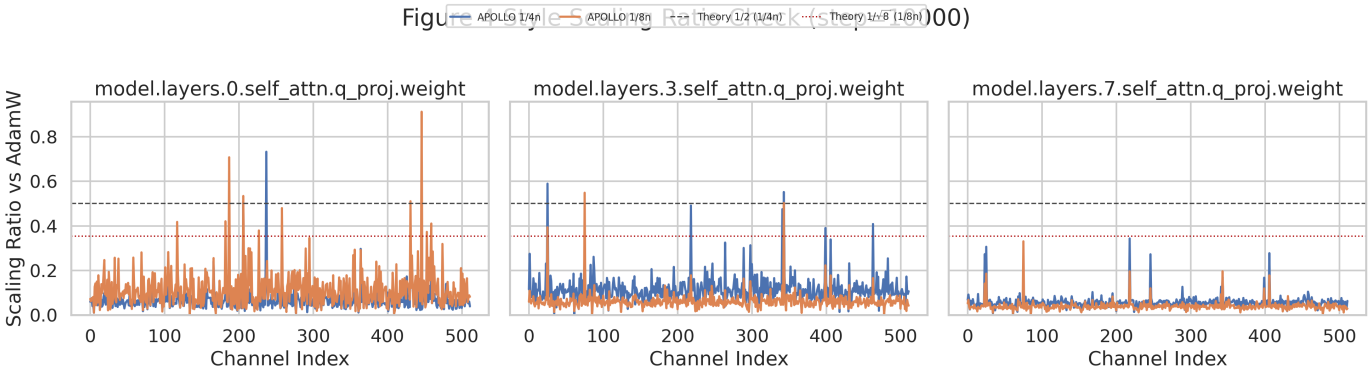


Figure 4: Scaling factor across multiple layers (start, middle, end of network) compared with the theoretic baseline on Llama-60M.

across layers, gradients in different channels behave similarly enough that sharing a scale is approximately lossless. Standard NanoGPT uses LayerNorm and learned positional embeddings, which may produce gradients whose useful anisotropy is more parameter-local: the element-wise scaling in AdamW carries optimization information that coarse grouping discards. This is consistent with prior work suggesting that adaptive learning rates are especially important for Transformers precisely because different parameter groups exhibit different Hessian spectra [2].

We also attempted to reproduce the scaling-ratio diagnostic from the original paper. Our mechanistic sanity check on the factor ratio s_j^R/s_j revealed significant discrepancies compared to the original results. Specifically, the observed ratio approached zero, indicating substantial information loss during the random projection process. We attribute this divergence to our reduced model scale and limited training duration: the theoretical bound of the projection is asymptotic in both time (t) and rank (r). At a 60M parameter scale with shortened runs, the approximation has insufficient opportunity to stabilize. Future work could explore dynamic update schedules for the random projections to better capture the gradient's spectral energy within the low-rank subspace.

5 Conclusion

We did validate APOLLO's core idea in a reduced setting: on LLaMA-60M + TinyStories, APOLLO rank-1/4 and rank-1/8 reduce optimizer-state memory while remaining close to AdamW in validation perplexity. We also discovered that this behavior is not universal. Taken together, the results suggest that APOLLO is best understood as a conditional optimizer: it works well when the architecture produces gradient statistics that are amenable to low-rank structured approximation, and it fails otherwise. Based on this, it is interesting to explore why LLaMA is more fitted to APOLLO. Following the feedback from the poster session, it would be useful to test LLaMA-specific components, like RMSnorm, SwiGLU, or RoPE inside other architectures in the hopes to explore what works best with APOLLO's hypothesis.

A Training curves of NanoGPT-130M + Wikitex

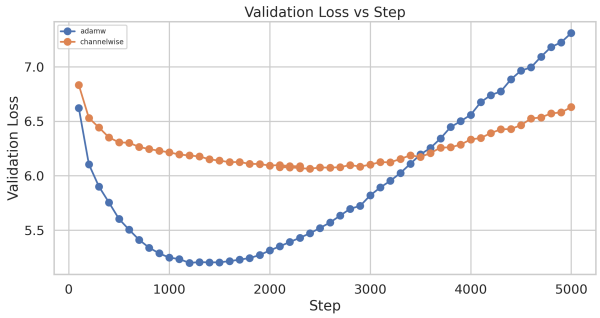


Figure 5: nanogpt-130M + wikitex with element-wise and channel-wise

References

[1] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[2] Yuxin Zhang, Chen Chen, Ting Ding, et al. Why Transformers Need Adam: A Hessian Perspective. *arXiv preprint arXiv:2402.16788*, 2024.

[3] Zhen Zhang, Hanqing Lu, et al. Adam-mini: Use Fewer Learning Rates to Gain More. *arXiv preprint*, 2024.

[4] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, et al. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. *ICML*, 2024.

[5] David Z. Pan, Hanqing Lu, Zhangyang Wang, and Jinwon Lee. APOLLO: SGD-like Memory, AdamW-level Performance. *Proceedings of MLSys*, 2025.

[6] Xi Chen, Kaituo Feng, Changsheng Li, Xunhao Lai, Xiangyu Yue, Ye Yuan, and Guoren Wang. Fira: Can We Achieve Full-rank Training of LLMs Under Low-rank Constraint? *arXiv preprint arXiv*, 2025.

[7] Dimitris Achlioptas. Database-friendly random projections: Johnson-Lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003.

[8] William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.